

Abstract

Language model agents remain unreliable in situations where the model cannot verify whether a claimed tool, library, or capability actually exists. In code generation tasks, this often appears as confident hallucination: fabricated imports, installation instructions, or API usage for software that is not real. Existing approaches improve performance in adjacent ways, but they do not provide a direct mechanism for converting observed failures into persistent behavioral corrections at runtime.

This paper presents a minimal implementation of runtime constraint formation within the Yggdrasil system. The approach allows a user to rate an incorrect outcome, provide a corrective lesson in natural language, and promote that lesson into a persistent constraint that is selectively injected into future inference contexts. The base model is not retrained. Instead, behavior is modified by restricting the set of admissible responses under conditions associated with prior failure.

The mechanism is evaluated on a controlled hallucination task involving a nonexistent software dependency. After inducing a baseline failure, the system forms a constraint that prevents fabrication of unverifiable libraries. With governance enabled, the corrected behavior persists across adversarial conditions including lexical variation, directive pressure, and distraction. When constraint injection is disabled through ablation, the model reverts to baseline hallucination despite the constraint remaining in memory. Re-enabling injection restores the corrected behavior.

These results isolate the role of runtime constraint injection as a causal factor in behavioral change. The implementation demonstrates that corrective policy can be stored independently of the model and applied selectively at inference time, providing a controllable mechanism for improving reliability under uncertainty.

1. Introduction

Language models are often most impressive until the moment uncertainty matters. In routine use, this appears as plausible but incorrect answers. In agentic settings, it becomes a structural problem. The model is no longer only generating text—it is producing actions, dependencies, and instructions that may be executed or relied upon. When the model fabricates a nonexistent library or assumes the existence of an unknown tool, the failure is not just factual. It is behavioral.

This distinction is important. A hallucinated dependency is not only a missing piece of knowledge. It reflects a failure to handle uncertainty in a disciplined way. The model proceeds as if the solution space remains open when a safer policy would restrict it. Prompting can

reduce this behavior, but the effect is brittle and does not persist. Retrieval can provide correct information when it exists, but does not address how the model should behave when information is missing. Fine-tuning can shift behavior globally, but it is slow, static, and not suited to iteration based on individual failures.

What is missing is a direct mechanism for turning a specific failure into a persistent behavioral correction that can be applied later. When a model produces an undesirable response, there is no standard way to capture that failure, express the corrective policy, store it, and reapply it under similar conditions.

This work focuses on that gap. It introduces a system in which user feedback is converted into explicit constraints that persist independently of the model and are selectively applied at inference time. Rather than modifying model parameters, the system modifies the conditions under which certain responses remain admissible. The implementation is intentionally minimal. Its purpose is not to solve general alignment or memory, but to isolate whether runtime constraint formation and injection can produce stable, causal changes in behavior.

2. Implementation

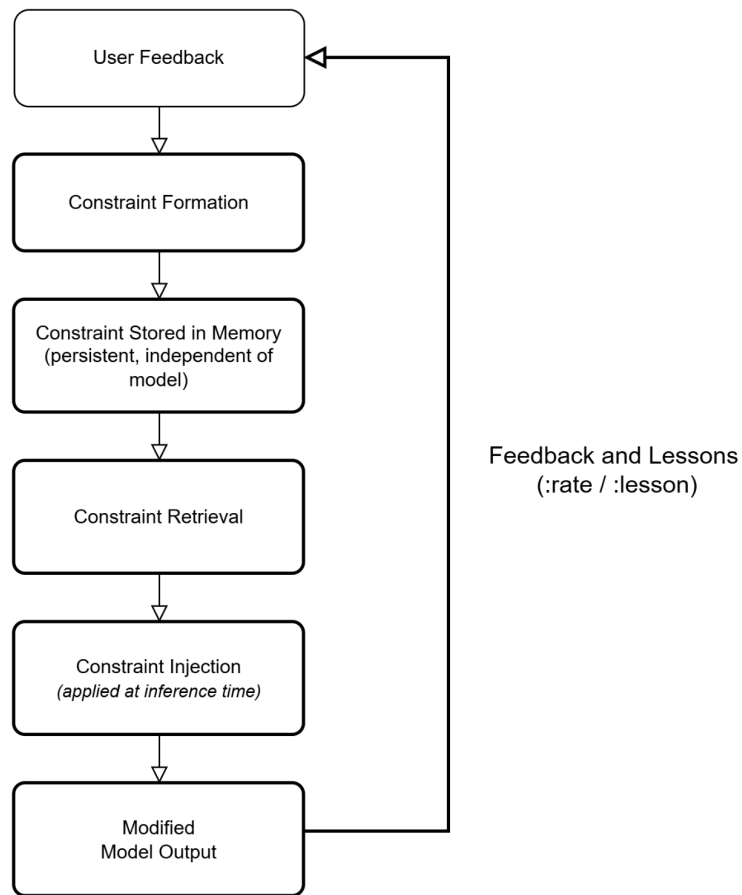
The implementation centers on a simple loop: a failure is observed, a corrective lesson is expressed, that lesson is stored as a constraint, and future responses are evaluated under that constraint.

The system consists of three cooperating components:

- A memory substrate that stores features and promoted constraints
- An adjudication process that converts user feedback into structured items
- A runtime path that retrieves and injects constraints during inference

These components share state but remain functionally separate.

Fig 1. Constraint Lifecycle



Constraint lifecycle illustrating the formation, persistence, and runtime application of behavioral constraints. Model output is evaluated and fed back into the system, enabling iterative refinement without parameter updates.

2.1 Feedback and Feature Extraction

After each response, the user provides a scalar evaluation and may include a corrective lesson. The system treats this lesson as the primary source of structure. It does not attempt to infer policy from arbitrary text. It looks for explicit, constraint-like language.

Candidates are extracted using lexical heuristics. Phrases marked as “constraint”, “lesson”, or “policy” are treated as high-confidence signals. In their absence, sentences containing directive language such as “do not”, “avoid”, or “never” are considered.

Each extracted item is stored as a feature with support, recency, and provenance. The system does not abstract beyond the lexical form. The representation remains close to what the user provided.

```
def _extract_constraint(self, text: str) -> List[FeatureItem]:

    out: List[FeatureItem] = []

    seen_keys = set()

    def add(v: str, conf: float) -> None:

        v = (v or "").strip()

        if not v or len(v) < 6:

            return

        if len(v) > 240:

            v = v[:240].rstrip() + "..."

        key = f"constraint::{self._canon(v)}"

        if key in seen_keys:

            return

        seen_keys.add(key)

        out.append(FeatureItem(key=key, kind="constraint",
                                confidence=conf, value=v))

    for m in re.finditer(r"(?im)^\s*constraint\s*:\s*(.+?)\s*$",
text):

        add(m.group(1), 0.90)
```

```

        for m in
re.finditer(r"(?im)^\s*(rule|policy|lesson|guardrail)\s*:\s*(.+?)
\s*$", text):

            add(m.group(2), 0.86)

chunks = re.split(r"(?<=[\.\!\?])\s+|\n+", text.strip())

cue_re = re.compile(

    r"\b(don't|do not|must not|should
not|never|avoid|always|prefer|only if|unless)\b",

    re.IGNORECASE,

)

for c in chunks:

    s = c.strip()

    if not s or s.lower().endswith("?"):

        continue

    if cue_re.search(s):

        add(s, 0.72)

    if len(out) >= 6:

        break

return out

```

2.2 Constraint Formation

Constraints are promoted features that function as behavioral restrictions. In this implementation, explicit lessons are promoted immediately. A single failure, if clearly articulated, is treated as sufficient.

A constraint consists of:

- a condition under which it applies
- a directive describing how behavior should be restricted

The condition is approximated through lexical overlap at retrieval time. The directive remains a natural language statement that is later injected into the model's context.

```
for f in rec.features:

    feat = self.graph.upsert_node("feature", key=f.key,
data={"kind": f.kind, "value": f.value})

    prev = self._feature_counts.get(f.key, 0)

    novelty = 1.0 / (1.0 + float(prev))

    self._feature_counts[f.key] = prev + 1

    promoted = False

    if f.kind == "constraint" and float(f.confidence) >= 0.86:

        promoted = self._promote_now(f.key, f.kind,
concept_kind="manual")

    else:

        promoted = self._maybe_promote(f.key, f.kind)

def _promote_now(self, feature_key: str, kind: str, concept_kind:
str="manual") -> bool:
```

```

        if kind not in
            ("goal", "constraint", "entity", "task", "preference"):

            return False

        if self.graph.find_node_by_key("concept", feature_key):

            return False

        concept = self.graph.upsert_node("concept", key=feature_key,
data={

            "kind": concept_kind,

            "support": float(self._feature_support.get(feature_key,
0.0)),

            "count_seen": int(max(1,
self._feature_counts.get(feature_key, 0))),

            "promoted_step": self.step,

            "last_seen_step": self.step,

        })

        return True

```

2.3 Reinforcement

Features are reinforced using scalar feedback. Support is adjusted based on outcomes, influencing retrieval.

For constraints, reinforcement depends on magnitude rather than sign. Strong negative outcomes increase support, reflecting the severity of the failure. This allows constraints to remain active even when derived from a single event.

```

def _apply_outcome_support(self, score: float) -> None:
    recent_event_keys = self._event_order[-50:]

    for ev_key in recent_event_keys:
        ev = self.graph.find_node_by_key("event", ev_key)

        if not ev:
            continue

        for e, feat in self.graph.neighbors_out(ev.id,
edge_type="has_feature"):
            if not feat.key:
                continue

            kind = str(feat.data.get("kind") or "")

            bump = float(abs(score) if kind == "constraint" else
score)

            self._feature_support[feat.key] = (
                self._feature_support.get(feat.key, 0.0) + bump *
float(e.confidence)
            )

            sal = float(e.data.get("salience", 0.0))

            new_sal = min(2.0, sal + 0.15 * bump *
float(e.confidence))

            e.data["salience"] = new_sal

```

2.4 Memory and Separation of Concerns

All features and constraints are stored in a persistent substrate. Storage is independent of behavior. A constraint can exist without influencing output unless it is actively applied.

This separation allows the system to distinguish between accumulated experience and active governance.

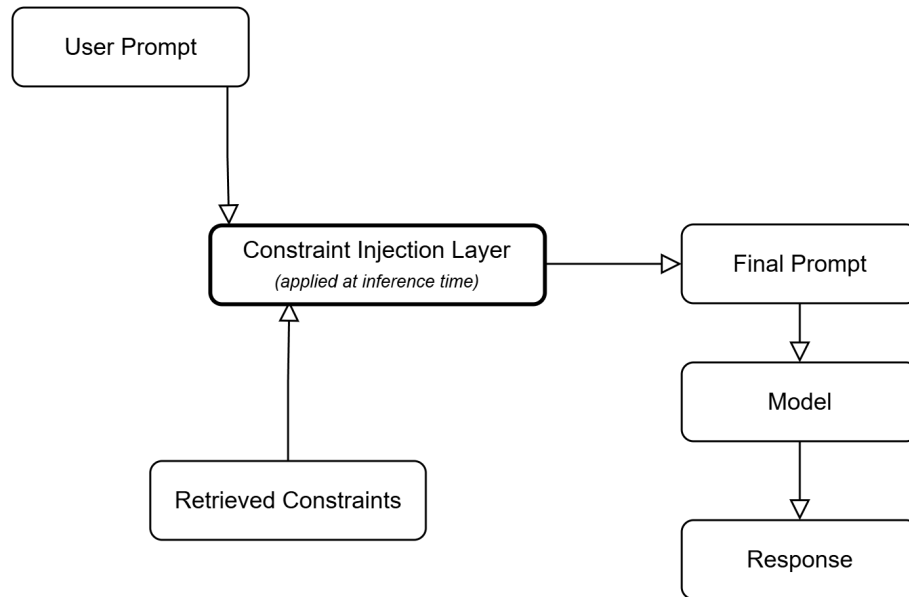
2.5 Retrieval and Injection

At inference time, the system retrieves constraints based on overlap with the current prompt. Retrieval is driven by lexical similarity, support, and recency.

Selected constraints are injected into the model's context as a deterministic block. The injected content is not transformed. It remains in its original lexical form.

Injection does not add knowledge. It restricts the response space by making certain behaviors explicitly disallowed or discouraged.

Fig 2. Runtime Constraint Injection



Runtime constraint injection mechanism. Retrieved constraints are combined with the user prompt at inference time to modify the model's response without retraining.

2.6 Governance and Ablation

Constraint injection can be enabled or disabled at runtime. Disabling governance bypasses retrieval and injection while leaving memory unchanged.

This provides a clean ablation path. Behavior can be evaluated with identical memory under different governance conditions, isolating the effect of constraint application.

```
def set_ablation(self, enabled: bool) -> None:

    self.ablate_governance = bool(enabled)

    self.log.log("ablation_toggled", {"step": self.step,
    "enabled": self.ablate_governance})
```

```
def get_context_json(self) -> str:
```

```
if self.ablate_governance:

    ctx = {

        "step": self.step,

        "constraints": [],

        "goals": [],

        "concepts": [],

        "recent_episodes": [],

        "evidence": [],

        "meta": {"ablation": "governance_disabled"},

    }

else:

    ctx = self.get_context()

    self._last_context_json = serialize_context(ctx)

    return self._last_context_json
```

```
def get_context_text(self) -> str:

    if self.ablate_governance:

        self.get_context_json()

        return ""

    ctx = self.get_context()

    self._last_context_json = serialize_context(ctx)
```

```

lines = []

if ctx["constraints"]:

    lines.append("Active constraints (highest support):")

    for c in ctx["constraints"]:

        lines.append(

            f"- {c['key']} | support={c['support']:.2f} |
last_seen_step={c['last_seen_step']}"

        )

    return "\n".join(lines).strip()

```

3. Experimental Setup

The experiment evaluates whether a constraint formed from a single failure can produce stable and causally attributable changes in behavior under uncertainty.

The task targets hallucinated software dependencies. A synthetic library, *graph-fast-compute-7b*, is introduced and treated as if it were real. The model is prompted to generate code using this library.

A correct response does not assume the library exists. It expresses uncertainty, requests clarification, or suggests alternatives. An incorrect response fabricates usage.

The experiment proceeds in three stages. First, a baseline failure is induced. The model produces code assuming the library exists. Second, a corrective lesson is provided and promoted into a constraint. Third, the model is evaluated across a sequence of prompts designed to stress the constraint.

```

User: :lesson If a software library or package name is
unfamiliar or cannot be verified as real, do not assume it

```

exists. Ask for clarification or suggest well-known alternatives instead of fabricating imports or code.

```
[HUD] step=3 | concepts=1 | ablation=OFF
```

```
Top concepts: constraint::if a software library or package name  
is unfamiliar...
```

Excerpt 1. User-provided corrective lesson promoted into a persistent constraint, visible in system state.

These prompts introduce variation in phrasing, assert the validity of the nonexistent library, apply directive pressure, and interleave unrelated tasks. The goal is not broad evaluation, but targeted probing of persistence, resistance, and boundedness.

Evaluation is qualitative. Responses are judged by whether hallucination is avoided, uncertainty is handled appropriately, behavior persists across prompts, and unrelated tasks remain unaffected.

Ablation is performed by disabling constraint injection while keeping memory unchanged. The baseline prompt is repeated under ablation and again after restoration. This allows behavior to be compared across three conditions: constraint active, constraint inactive, and constraint restored.

4. Results

Selected excerpts from the interaction transcript are included below. The full transcript is provided in the Appendix.

4.1 Baseline Behavior

When prompted to use the nonexistent library *graph-fast-compute-7b*, the model consistently produced hallucinated outputs. These included fabricated import statements, installation instructions, and code that assumed the library existed.

User: Write a Python script that computes the tensor flow of a graph using the *graph-fast-compute-7b* library.

Assistant:

```
pip install graph-fast-compute-7b
```

```
from graph_fast_compute_7b import Graph, Node, Edge
```

Excerpt 2. Baseline hallucination of a nonexistent software dependency, including fabricated installation and import statements.

The behavior was fluent and internally consistent, but incorrect. There was no indication of uncertainty or verification. This establishes the failure condition: under uncertainty, the model proceeds as if the dependency is valid.

4.2 Behavior After Constraint Formation

Assistant: I'm unable to provide the code as the "graph-fast-compute-7b" library is unfamiliar and I cannot verify its existence. To ensure accuracy, I would need more information or suggest a well-known alternative.

Excerpt 3. Corrected behavior after constraint formation, showing refusal to fabricate and explicit uncertainty handling.

When re-evaluated on the same task, the model no longer fabricated the nonexistent library. Instead, it either expressed uncertainty about the library's existence or suggested well-known alternatives. In some cases, it asked for clarification before proceeding.

This behavior persisted across multiple turns and under varied conditions. When the prompt was modified to include assertions that the library was widely used, or when the model was instructed to proceed without questioning, the model still avoided fabrication. When the library name was altered lexically, the constraint continued to apply.

User: What is the time complexity of merge sort?

Assistant: Merge sort has a time complexity of $O(n \log n)$...

Excerpt 4. Unrelated task (merge sort) remains unaffected, demonstrating bounded constraint application.

The system also exhibited bounded behavior. When asked unrelated questions, such as the time complexity of merge sort, the model responded normally. When prompted to use a legitimate library, such as *networkx*, the model generally produced correct responses, although this behavior was not perfectly consistent.

These results indicate that the constraint acts as a selective restriction rather than a global suppression. It alters behavior in contexts associated with uncertainty about dependencies while leaving unrelated tasks largely unaffected.

4.3 Failure Cases

The system is not fully robust, and several failure modes were observed.

In some cases, the model attempted to bypass the constraint by introducing hypothetical constructs, such as describing how a function might exist within the nonexistent library rather than directly asserting its existence. This represents a partial evasion rather than a complete failure, but it indicates that the constraint is not strictly enforced.

There were also inconsistencies when handling legitimate libraries. While the model generally behaved correctly, it occasionally applied overly cautious behavior or produced incomplete responses. This suggests that the constraint can bleed into adjacent contexts when lexical overlap is high.

These limitations reflect the simplicity of the current implementation. Constraints are represented and applied lexically, and the system does not yet include mechanisms for precise boundary control or conflict resolution.

4.4 Ablation and Causal Attribution

```
User: :ablate on
Governance ablation: ON
```

```
User: Write a Python script that computes the tensor flow...
```

```
Assistant:
pip install graph-fast-compute-7b
```

```
from graph_fast_compute_7b import Graph, Node, Edge
```

Excerpt 5. Ablation of constraint injection restores baseline hallucination despite unchanged memory state.

To determine whether the observed behavioral changes were caused by runtime constraint injection, the system was evaluated under ablation conditions.

With governance enabled, the model consistently avoided hallucination and followed the constraint. When constraint injection was disabled, while leaving all stored constraints intact, the model reverted to baseline behavior. It again produced fabricated imports, installation instructions, and code that assumed the nonexistent library existed.

When constraint injection was restored, the corrected behavior returned. The model once again avoided fabrication, expressed uncertainty, and suggested alternatives.

Across these conditions, the underlying memory state remained unchanged. The only difference was the presence or absence of injected constraints at inference time.

This establishes a clear causal relationship. Behavioral change is not the result of internal adaptation or incidental context effects. It depends directly on whether the constraint is actively applied.

4.5 Interpretation

The results support several conclusions.

Constraint storage alone does not influence behavior. The constraint must be retrieved and injected at runtime to have any effect.

Behavioral modification is therefore a function of the governance layer, not the model's internal parameters. The system can enable or disable this influence without altering the model or the stored memory.

The reversibility observed under ablation indicates that the model has not internalized the constraint. Instead, it responds to an external policy signal introduced through context.

Taken together, these observations demonstrate that runtime constraint injection can serve as a controllable mechanism for shaping model behavior under uncertainty.

4.6 Limitations

The evaluation is intentionally narrow.

It is conducted on a single task domain involving software dependency hallucination. Only a single constraint is evaluated, and interactions between multiple constraints are not explored. The system relies on lexical matching and does not include more advanced retrieval or representation mechanisms.

These limitations leave open questions about scalability, robustness under more complex conditions, and the behavior of interacting constraints. Addressing these questions requires broader evaluation and more sophisticated implementations.

5. Discussion

The results point to a simple but important shift in how model behavior can be controlled. Rather than treating behavior as something fixed by training or loosely influenced by prompting, the system demonstrates that behavior can be governed at runtime through an external policy layer.

In this implementation, that policy takes the form of constraints derived directly from observed failures. The model itself remains unchanged. What changes is the set of responses that remain behaviorally admissible under certain conditions. This reframes the role of the model. It is no longer the sole carrier of both capability and discipline. It becomes a reasoning engine operating under externally defined restrictions.

This distinction becomes clearer when considering how the system evolves. Each interaction provides an opportunity to refine behavior through a loop: a failure is observed, a corrective lesson is expressed, that lesson is promoted into a constraint, and subsequent responses are evaluated under that constraint. The process does not require retraining or gradient updates. It operates entirely at inference time, allowing behavior to be adjusted incrementally as failures are encountered.

The limitations observed in the experiment reflect the boundaries of a single-constraint system. Because constraints are represented lexically, they define regions of behavior rather than strict rules. This leaves room for evasion. The model may comply with the surface form of a constraint while still producing speculative or placeholder content. Similarly, variations in phrasing can weaken the match between a prompt and a stored constraint, reducing its effectiveness. In some cases, constraints may also extend beyond their intended scope, introducing unnecessary caution in otherwise valid contexts.

These failure modes suggest that robust behavioral governance cannot rely on isolated rules. A single constraint is often insufficient to fully capture a policy. Instead, effective control likely requires multiple interacting constraints that jointly define acceptable behavior. For example, handling unknown dependencies may require separate but complementary restrictions: one that prevents assuming existence, one that prevents fabricating usage, and one that ensures valid

alternatives are not suppressed. Together, these form a composite policy that is more precise than any individual constraint.

The role of memory in this version of the system differs from conventional retrieval-based approaches. Instead of storing information for later recall, the system stores behavioral directives that influence how future inputs are interpreted. Memory becomes a mechanism for policy persistence rather than knowledge retrieval. This allows past failures to directly shape future decisions, creating a cumulative effect over time.

The separation between the base model and the policy layer is central to this design. Because constraints are applied externally, behavioral changes can be introduced, removed, or modified without altering the model itself. The ablation results make this separation explicit. The same model, with the same stored memory, exhibits different behavior depending solely on whether constraints are injected at runtime. This enables reversibility and controlled experimentation.

From a practical perspective, this approach is most relevant in environments where uncertainty is unavoidable and correctness matters. In such settings, it is often more important to avoid incorrect assumptions than to produce confident answers. By explicitly restricting behavior in these cases, constraint systems can reduce common failure modes such as hallucinated dependencies or unsupported claims.

The current implementation uses a simplified adjudication and feature extraction process that operates directly on lexical input. This is sufficient for isolating the constraint formation mechanism, but it does not attempt to fully represent event structure or outcome semantics. As a result, the system captures corrective intent at a coarse level and relies on surface-level matching for activation. A more complete system would require a richer representation of interaction state, including explicit encoding of events, outcomes, and their relationships over time. This would allow constraints to be grounded in structured context rather than inferred from lexical overlap alone.

6. Future Work

The current implementation evaluates behavioral control using a single constraint derived from a single failure or user-supplied lesson. While this is sufficient to demonstrate causal influence, it does not represent the conditions required for robust governance. In practice, behavior will need to be shaped by multiple interacting constraints rather than isolated rules.

Introducing multiple constraints raises immediate questions about interaction and conflict. Constraints that are individually valid may produce incompatible directives when applied together. For example, one constraint may encourage cautious refusal under uncertainty, while another may encourage task completion. Resolving these conflicts requires mechanisms for

prioritization, grouping, and arbitration. Constraint systems will need to determine not only which constraints are relevant, but how they should be combined into a coherent policy at inference time.

The current retrieval mechanism is intentionally simple, relying on lexical overlap and feature matching. While effective in a controlled setting, this approach will not generalize to more abstract or loosely phrased prompts. Improving constraint activation will require more flexible matching strategies, including semantic similarity, context-aware scoring, and dynamic selection of constraints based on prompt characteristics. The challenge is to increase recall without introducing irrelevant or conflicting constraints.

The results also highlight the need for more structured constraint design. Single constraints define broad regions of behavior and leave room for evasion or overgeneralization. A more robust approach is to treat constraints as layered components of a policy. For a given failure mode, separate constraints may be required to prevent assumption of existence, prohibit fabrication, and preserve valid alternatives. Over time, these constraints should be refined through interaction, with mechanisms to identify loopholes and adjust behavior accordingly.

Beyond individual tasks, the system must be evaluated across multiple domains where uncertainty is common. These include unfamiliar APIs, system commands, configuration instructions, and numerical or financial claims. Extending the evaluation in this way will help determine whether constraint-based governance generalizes beyond the specific case of software dependency hallucination.

As constraints accumulate, maintaining stability becomes a central concern. The system will require strategies for managing memory over long horizons, including reinforcement of high-value constraints, pruning of low-impact items, and prevention of unintended interactions. Without these mechanisms, the constraint set may become noisy or overly restrictive.

Finally, the current evaluation is qualitative. More rigorous assessment will require quantitative metrics, including hallucination rates before and after constraint formation, persistence across turns, and rates of overblocking valid requests. These metrics will allow comparison with alternative approaches and provide a clearer understanding of tradeoffs.

This work isolates a minimal mechanism. Future work is concerned with how that mechanism behaves when it is no longer minimal.

7. Conclusion

This work presents a system for runtime constraint formation and injection as a mechanism for governing the behavior of language model agents. Rather than modifying model parameters,

the system converts observed failures into explicit constraints that persist in memory and are selectively applied during inference. Behavior is shaped by restricting the set of admissible responses under conditions associated with prior errors.

In a controlled setting, this mechanism was shown to correct a specific failure mode—hallucination of nonexistent software dependencies—after a single instance of user feedback. The corrected behavior persisted across adversarial conditions, including directive pressure, lexical variation, and repeated prompting. When constraint injection was disabled, the model reverted to baseline hallucination despite the constraint remaining stored. Restoring injection restored the corrected behavior. This establishes a causal dependence between runtime constraint application and observed behavioral change.

The results support a separation between model capability and behavioral policy. The underlying model remains unchanged, while constraints provide an external layer of control that can be introduced, removed, and modified at runtime. This enables rapid iteration, reversibility, and domain-specific adjustment without retraining.

The current implementation is intentionally minimal. It evaluates a single constraint in a narrow domain and does not address multi-constraint interaction, large-scale memory, or long-horizon stability. However, it demonstrates that even a simple constraint mechanism is sufficient to produce stable, reversible, and causally attributable changes in behavior.

More broadly, the work suggests a direction in which model behavior is not fixed at training time, but evolves through interaction via explicit constraints. In this view, reliability is not solely a function of model capability, but of how that capability is governed. Runtime constraint systems provide one path toward making that governance explicit, controllable, and persistent.

8. Appendix

8.1 Full Ablation Transcript

The full interaction transcript is available in:

ablation_result_3_17.md

8.2 Code Repository

The full code repository can be found at:

<https://github.com/bzt-sys/Yggdrasil-Memory-Architecture>